SOFTWARE REQUIREMENTS SPECIFICATION

Student Record Management System

Team-H11

Name and SRN: Shriranjini-PES1UG24CS447

Shantheri Shenoy-PES1UG24CS428

Spoorthi T-PES1UG24CS462

Document Control

Field	Value
Project	Student Record Management System
Team	H-11
Document	Updated Software Requirements Specification (SRS)
Baseline	Phase 1: menu-driven C application with CSV persistence
Current version	Updated dual-track implementation: C99 console engine + Flask/MySQL web application
Status	Updated from implementation summary supplied by the project team
Evaluation evidence	53 automated unit tests reported passed; 100% pass rate; 82% statement and branch coverage reported

Revision note: This edition incorporates the Phase 2/Phase 3 changes described by the team. The Phase 1 source list containing the original exact wording of FR-01 to FR-25 and NFR-01 to NFR-12 was not supplied. Therefore, the requirement titles and descriptions below are a normalized traceability mapping reconstructed from the stated scope and implementation delta. Replace these labels with the approved baseline wording if exact historical wording must be preserved.

1. Introduction

1.1 Purpose

This SRS specifies the functional, non-functional, and technical requirements of the Student Record Management System (SRMS). It defines the system's dual-track operation, user roles, student records, assessment and attendance workflows, data persistence, security controls, and quality requirements. It is intended for the project team, faculty evaluators, testers, maintainers, and other stakeholders.

1.2 Scope

SRMS manages student profiles, user accounts, subject-level faculty access, continuous assessment marks, attendance, and supporting audit information. It retains a C ISO C99 console engine for low-level record operations, integrity algorithms, file reliability, backup/restore, and performance benchmarking. A web application built with Python 3.13, Flask, MySQL 8.0, HTML5, CSS3, and vanilla JavaScript provides browser-based workflows through a REST-style backend.

1.3 Definitions and Acronyms

Term	Meaning
SRMS	Student Record Management System
FR / NFR / MTR	Functional / Non-Functional / Mandatory Technical Requirement
RBAC	Role-Based Access Control
ISA1 / ISA2 / ESA	Internal assessment 1 / Internal assessment 2 / End-semester assessment
CSRF	Cross-Site Request Forgery
ACID	Atomicity, Consistency, Isolation, Durability
CSV	Comma-Separated Values

C99	ISO C programming language standard (1999)

1.4 References and Conventions

This document follows the section structure commonly used in IEEE-style SRS documents (purpose, scope, overall description, specific requirements, interfaces, and traceability). Requirement IDs are stable identifiers. 'Shall' denotes a mandatory requirement. Verification statuses in the matrix reflect the team's supplied evaluation summary; individual test-case-to-requirement links were not supplied.

2. Overall Description

2.1 Product Perspective

The product has two parallel implementation tracks. The C track is a standalone console engine using CSV files. The web track is a Flask application backed by a normalized MySQL database. They share the student-record domain but use distinct persistence mechanisms; the supplied design does not establish live synchronization between CSV files and MySQL. The web application is available locally at http://127.0.0.1:5000.

2.2 Product Functions

- Create, view, search, and maintain student records.
- Manage administrator, faculty, and student accounts and enforce role-based permissions.
- Assign faculty to specific course, semester, and subject combinations.
- Maintain curriculum mappings for four departments and eight semesters.
- Record ISA1, ISA2, and ESA marks, enforce administrator-controlled grade windows, and audit mark changes.
- Record attendance, calculate attendance percentages, and flag values below 75%.
- Preserve C-engine CSV data through duplicate checks, atomic saves, snapshots, and restore operations.
- Provide secure web requests, password hashing, CSRF protection, and browser security headers.

2.3 User Classes

User class	Responsibilities / permissions
Administrator	Manage users and student records; configure grade windows; create faculty assignments; oversee system data.
Faculty	View only assigned subject/semester class cards and rosters; enter or edit marks only while the relevant grade window is open; provide a reason for edits.
Student	View only the student's own linked record and permitted academic information; no access to other students' records.
Maintainer / tester	Operate the C engine, configure the local web stack, run tests, and perform backup/recovery and performance checks.

2.4 Operating Environment

- C track: ISO C99 compiler and a supported desktop operating system; CSV files students.csv, users.csv, grades.csv, and attendance.csv.
- Web track: Python 3.13, Flask, Werkzeug password hashing, MySQL 8.0 with InnoDB, HTML5, CSS3, and vanilla JavaScript.
- Web development/runtime endpoint: http://127.0.0.1:5000.
- Database: student_records schema.

2.5 Design and Implementation Constraints

- The C engine and web application are separate execution tracks; CSV and MySQL are not assumed to be automatically synchronized.
- MySQL foreign keys and InnoDB transactional constraints shall be used for relational integrity.
- Passwords shall not be stored in plaintext.
- Faculty permissions are scoped by explicit subject/course/semester assignments.
- Grade entry and editing are governed by inclusive administrator-defined date windows.
- The system shall retain the specified C99 compatibility for the console engine.

2.6 Assumptions and Dependencies

- MySQL is available and configured with the student_records schema before web workflows are used.
- Administrators maintain correct faculty assignments, curriculum mappings, and grade-window dates.
- System date/time is configured correctly because grade-window enforcement is date-dependent.
- Users access the web application through a supported modern browser.
- The supplied evaluation figures and implementation descriptions are accurate team-reported evidence.

3. System Features and Functional Requirements

ID	Requirement	Specification
FR-01	User authentication	The web application shall authenticate users using registered credentials and establish an authenticated session.
FR-02	Role-based authorization	The system shall distinguish admin, faculty, and student roles and authorize actions according to role.
FR-03	Student record creation	An authorized administrator shall be able to create a student record with student ID, name, contact details, course, and semester.
FR-04	Student record retrieval	Authorized users shall be able to retrieve student information permitted by their role and assignment.
FR-05	Student record update	An authorized administrator shall be able to update student profile fields.
FR-06	Student record deletion/deactivation	The system shall provide an authorized administrative mechanism to remove or deactivate records, subject to referential-integrity constraints.
FR-07	Student search	The system shall support student lookup by student identifier and/or supported student attributes.
FR-08	Duplicate student-ID prevention	The C engine shall reject duplicate student IDs without regard to letter case, both during in-memory insertion and before file persistence.
FR-09	User account management	An administrator shall be able to create and manage user accounts with a defined role and, where applicable, a linked student record.
FR-10	Faculty assignment management	An administrator shall assign faculty users to specific course, semester, and subject combinations.
FR-11	Faculty class-card visibility	Faculty shall see only class cards corresponding to their active assignments.
FR-12	Faculty roster visibility	Faculty shall retrieve student rosters only for assigned subject/course/semester combinations.
FR-13	Curriculum mapping	The system shall maintain course/semester/subject mappings for the supported four departments and eight semesters.
FR-14	Assessment component management	The system shall support separate ISA1, ISA2, and ESA assessment components, each recorded on a 0–100 mark scale.
FR-15	Grade-window configuration	An administrator shall configure inclusive start and end dates for each course, semester, and assessment component.
FR-16	Grade-window enforcement	The system shall block faculty mark creation or modification when the applicable grade window is closed and return HTTP 403 for blocked web requests.
FR-17	Grade entry	Authorized faculty shall enter marks for students within their assigned scope while the relevant grade window is open.
FR-18	Grade editing with reason	A faculty mark edit shall require a non-empty reason and shall create an audit record containing old marks, new marks, reason, actor, and timestamp.
FR-19	Grade viewing	Authorized users shall view assessment marks permitted by their role and student/subject scope.
FR-20	Attendance entry and update	Authorized users shall record attended classes and total classes for a student and subject.
FR-21	Attendance validation and calculation	The system shall reject negative or logically invalid attendance values, including attended

		classes greater than total classes, and calculate percentage automatically.
FR-22	Attendance shortage flag	The system shall flag attendance percentages below 75%.
FR-23	CSV persistence and recovery	The C engine shall load and save its CSV datasets and support snapshot backup and restore of primary files.
FR-24	Atomic file save	The C engine shall save through a temporary file and safe replacement sequence, restoring the prior file if replacement fails.
FR-25	Audit and integrity persistence	The web application shall persist grade-change audit history and enforce relational integrity through database keys and constraints.

4. External Interface Requirements

4.1 User Interface

The web interface shall be browser-based and implemented using HTML5, CSS3, and vanilla JavaScript. It shall present role-appropriate navigation and views. Faculty views shall expose assigned class cards and rosters only; student views shall be restricted to the authenticated student's own information. The C track shall retain its menu-driven console interface.

4.2 Software Interfaces

Interface	Description
Flask backend	backend/app.py exposes web application routes / REST-style operations.
Web client	Files under web/ provide the browser UI and communicate with the Flask backend.
MySQL	MySQL 8.0 schema student_records; InnoDB tables and foreign-key relationships.
C file interface	CSV files: students.csv, users.csv, grades.csv, attendance.csv.
Runtime endpoint	Local development endpoint http://127.0.0.1:5000.

4.3 Communications Interfaces

The web client communicates with the Flask backend over HTTP in the local deployment described by the team. Mutating requests use POST, PUT, or DELETE and shall include a valid CSRF token. Deployment beyond localhost requires an appropriately secured transport configuration.

5. Data Requirements

5.1 Database Entities

Table	Key fields / purpose
students	student_id (PK), name, contact_details, course, semester, created_at.
users	user_id (PK), username (UNIQUE), password_hash, role ENUM(admin, faculty, student), student_id (FK).
grades	grade_id (PK), student_id (FK), subject, component ENUM(ISA1, ISA2, ESA), marks INT, updated_by (FK), reason.
grade_windows	course, semester, component, start_date, end_date; composite primary key.
grade_change_history	old_marks, new_marks, reason, changed_by, changed_at; append-oriented audit history.
attendance	attendance_id (PK), student_id (FK), subject, attended_classes, total_classes, percentage.
faculty_assignments	user_id (FK), course, semester, subject; defines faculty scope.
course_semester_subjects	Curriculum mapping across four departments and eight semesters.

The supplied dataset contains 646 student records across Business Administration, Computer Science, Electronics, and Information Technology, spanning eight semesters.

5.2 Data Validation and Integrity

- Student IDs in the C engine are compared case-insensitively to prevent duplicates such as S001 and s001.
- Attendance input shall satisfy 0 ≤ attended_classes ≤ total_classes; percentage is derived from the counts. A zero total requires a defined safe handling rule (recommended: percentage unavailable/0 with no division by zero).
- Assessment marks shall be integers in the range 0–100.
- Foreign keys shall prevent orphaned student, user, grade, and assignment references.
- Grade edits shall be accompanied by a reason and an audit entry.

6. Non-Functional Requirements

ID	Quality attribute	Specification
NFR-01	Usability	The console and web interfaces shall provide understandable, role-appropriate workflows and feedback.
NFR-02	Performance	For the stated 1,000-record C-engine stress test, save shall complete within 1.0 s, load within 2.0 s, and search within 1.0 s.
NFR-03	Data integrity	The C engine shall prevent case-insensitive duplicate student IDs during insertion and file save; relational data shall use database constraints.
NFR-04	Security – credential protection	Passwords shall be stored only as cryptographic hashes using Werkzeug/scrypt; newly created users shall have passwords of at least 12 characters.
NFR-05	Security – request protection	All mutating web requests shall validate a CSRF token.
NFR-06	Backup and recovery	The C engine shall support snapshot backup and restore of corrupted primary CSV files.
NFR-07	Data reliability / crash safety	CSV replacement shall use a temporary-file and rollback-capable replacement sequence so a failed save preserves the previous valid file.
NFR-08	Access control and privacy	The application shall enforce role, subject, semester, and student ownership restrictions; students shall not access another student's record.
NFR-09	Web security headers	Responses shall include X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN, and a Content-Security-Policy.
NFR-10	Auditability	Grade changes shall be traceable to the prior value, new value, reason, actor, and timestamp.
NFR-11	Scalability	The C engine shall meet the stated performance targets at 1,000 student records; the relational web track shall support the supplied 646-record dataset and normalized growth.
NFR-12	Maintainability / portability	The C engine shall conform to ISO C99; the web track shall use the specified Python/Flask/MySQL/web stack with separated backend and web UI components.

7. Technical / Mandatory Technical Requirements

ID	Technical requirement
MTR-01	The console engine shall be implemented in ISO C99 and retain its menu-driven operation and CSV file interface.
MTR-02	The web application shall use Python 3.13 with Flask, MySQL 8.0/InnoDB, and an HTML5/CSS3/vanilla JavaScript frontend.
MTR-03	The web data model shall implement the specified student, user, grade, grade-window, grade-history, attendance, faculty-assignment, and curriculum-mapping entities with appropriate primary keys, unique constraints, and foreign keys.

8. Security Requirements and Rules

- Passwords must never be persisted in plaintext; Werkzeug scrypt hashes are stored in users.password_hash.

- Newly created user passwords must be at least 12 characters.
- All state-changing POST/PUT/DELETE requests must pass CSRF validation.
- Responses must set the three specified security headers.
- Authorization must be enforced server-side; hiding UI controls alone is insufficient.
- Student access must be checked against the authenticated user's linked student_id. The implementation summary phrases this as student_id == session.user_id; because users.user_id and users.student_id are distinct schema fields, the implementation should compare the requested student identifier to the authenticated session's linked student_id (or document an explicit session mapping).
- Faculty access must be validated against faculty_assignments for the requested course, semester, and subject.
- Grade-window dates are inclusive. Requests outside the active window must be rejected; the supplied implementation returns HTTP 403.
- Every mark edit must have a reason and corresponding grade_change_history record.

9. Verification, Validation, and Acceptance

The team reports 53 automated unit tests with a 100% pass rate and 82% statement and branch coverage. The C engine was stress-tested with 1,000 records: save 0.075 s, load 0.111 s, and search under 0.0001 s. These results satisfy the stated NFR-02/NFR-11 thresholds. Grade-window rejection, role scoping, student ownership checks, attendance validation, CSRF protection, password hashing, security headers, and audit logging are specified as implemented in the supplied project summary.

Verification status in the RTM is marked VERIFIED / PASSED as requested and based on the aggregate evaluation supplied by the team. The supplied summary does not include the 53 individual test names, test-to-requirement mapping, test environment, or coverage report; those artifacts should be attached to the submission if an evaluator requires independently reproducible evidence.

9.1 Performance Results

Operation	Measured at 1,000 records	Target	Result
Save	0.075 s	< 1.0 s	PASS
Load	0.111 s	< 2.0 s	PASS
Search	< 0.0001 s	< 1.0 s	PASS

10. Requirements Traceability Matrix (RTM)

All 25 functional requirements and all 12 non-functional requirements are shown as VERIFIED / PASSED, reflecting the reported 53/53 automated test outcome. MTR verification is also included for completeness.

Req. ID	Requirement area	Status	Verification evidence
FR-01	User authentication	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-02	Role-based authorization	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-03	Student record creation	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-04	Student record retrieval	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-05	Student record update	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-06	Student record deletion/deactivation	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-07	Student search	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-08	Duplicate student-ID prevention	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-09	User account management	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-10	Faculty assignment management	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-11	Faculty class-card visibility	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-12	Faculty roster visibility	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-13	Curriculum mapping	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-14	Assessment component management	VERIFIED / PASSED	Functional test suite; aggregate result reported

FR-15	Grade-window configuration	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-16	Grade-window enforcement	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-17	Grade entry	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-18	Grade editing with reason	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-19	Grade viewing	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-20	Attendance entry and update	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-21	Attendance validation and calculation	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-22	Attendance shortage flag	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-23	CSV persistence and recovery	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-24	Atomic file save	VERIFIED / PASSED	Functional test suite; aggregate result reported
FR-25	Audit and integrity persistence	VERIFIED / PASSED	Functional test suite; aggregate result reported
NFR-01	Usability	VERIFIED / PASSED	Automated tests / stress and security checks as applicable; aggregate result reported
NFR-02	Performance	VERIFIED / PASSED	Automated tests / stress and security checks as applicable; aggregate result reported
NFR-03	Data integrity	VERIFIED / PASSED	Automated tests / stress and security checks as applicable; aggregate result reported
NFR-04	Security – credential protection	VERIFIED / PASSED	Automated tests / stress and security checks as applicable; aggregate result reported
NFR-05	Security – request protection	VERIFIED / PASSED	Automated tests / stress and security checks as applicable; aggregate result reported
NFR-06	Backup and recovery	VERIFIED / PASSED	Automated tests / stress and security checks as applicable; aggregate result reported
NFR-07	Data reliability / crash safety	VERIFIED / PASSED	Automated tests / stress and security checks as applicable; aggregate result reported
NFR-08	Access control and privacy	VERIFIED / PASSED	Automated tests / stress and security checks as applicable; aggregate result reported
NFR-09	Web security headers	VERIFIED / PASSED	Automated tests / stress and security checks as applicable; aggregate result reported
NFR-10	Auditability	VERIFIED / PASSED	Automated tests / stress and security checks as applicable; aggregate result reported
NFR-11	Scalability	VERIFIED / PASSED	Automated tests / stress and security checks as applicable; aggregate result reported
NFR-12	Maintainability / portability	VERIFIED / PASSED	Automated tests / stress and security checks as applicable; aggregate result reported
MTR-01	Technical stack / schema compliance	VERIFIED / PASSED	Implementation review; team-reported
MTR-02	Technical stack / schema compliance	VERIFIED / PASSED	Implementation review; team-reported
MTR-03	Technical stack / schema compliance	VERIFIED / PASSED	Implementation review; team-reported

11. Appendix A – Implementation Architecture Summary

- Presentation layer: C menu-driven console; web browser UI (HTML/CSS/JavaScript).
- Application layer: C engine orchestration; Flask routes, session handling, and application services.
- Validation / Logic layer: C duplicate detection, record operations, atomic-save and backup logic; Flask RBAC, faculty assignment checks, grade-window rules, attendance validation/calculation, CSRF checks, and audit logic.
- Data layer: CSV files for the C track; MySQL 8.0/InnoDB relational tables for the web track.

12. Appendix B – Known Clarifications for Final Submission

- Confirm the exact Phase 1 wording and titles of FR-01…FR-25 and NFR-01…NFR-12 against the original SRS; the IDs in this updated edition are a reconstructed mapping because the original requirement catalogue was not included in the supplied summary.
- Attach a test report mapping each requirement ID to one or more of the 53 automated tests, plus the coverage report, to substantiate individual VERIFIED statuses.

- Clarify whether CSV and MySQL data are intentionally independent or whether a synchronization/import/export mechanism exists.
- Confirm the student session ownership check uses the user's linked student_id rather than directly comparing a student_id to a user_id.
- Specify the exact policy for attendance when total_classes is zero and the deletion behavior for records referenced by foreign keys.